Name: Saran.S

Reg no: 192473003

Course name: CSA0402 – Java programming

C02 AT2 ASSESSMENT

Scenario Based

Q1 Smart University Payroll Management System 25 marks

A university employs three categories of staff: Teaching Faculty, Technical Staff, and Visiting Faculty. The payroll system should calculate the monthly salary based on the employee type while maintaining a common employee hierarchy. Requirements Create a superclass Employee with the following private data members: employeeId employeeName department basicSalary Provide: Default constructor Parameterized constructor Getter and Setter methods displayEmployee() calculateSalary() Create the following subclasses: TeachingFaculty teachingHours incentivePerHour Salary: basicSalary + (teachingHours × incentivePerHour) TechnicalStaff overtimeHours overtimeRate Salary: basicSalary + (overtimeHours × overtimeRate) VisitingFaculty lecturesHandled paymentPerLecture Salary: lecturesHandled × paymentPerLecture Override calculateSalary() in all subclasses. Create an array of Employee objects and demonstrate runtime polymorphism. Display: Employee details Monthly salary Highest-paid employee Average salary Search an employee using the employee ID.

Test Case 1 (Normal Case)
Input
3

TeachingFaculty
101
Dr. Arun
CSE

Your Code

```java
import java.util.*;

public class Main {
    static ArrayList<String> patientIds = new ArrayList<>();
    static HashMap<String, Patient> patientMap = new HashMap<>();
    static HashMap<String, Integer> deptCount = new HashMap<>();
    static final int MAX_PATIENTS = 100;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        boolean running = true;

        while (running) {
            int choice = Integer.parseInt(sc.nextLine().trim());

            switch (choice) {
                case 1:
                    registerPatient(sc);
                    break;
                case 2:
                    searchPatient(sc);
                    break;
                case 3:
                    displayAllPatients();
                    break;
                case 4:
                    departmentWiseCount();
                    break;
                case 5:
                    running = false;
                    break;
                default:
                    System.out.println("Invalid Choice");
            }
        }
    }

    static void registerPatient(Scanner sc) {
        if (patientIds.size() >= MAX_PATIENTS) {
            System.out.println("Maximum Patient Limit Reached.");
            return;
        }

        String id = sc.nextLine().trim();

        if (patientMap.containsKey(id)) {
```

Input

```
1
P101
Rahul Kumar
9876543210
```

Output

```
Patient Registered Successfully.
Patient Registered Successfully.
--------------------------------
--------------------
ID Name Department
--------------------------------
```

Visible Test Cases

- Test Case 1
- Test Case 2
- Test Case 3

0.00 / 20 marks

```
        String mobile = sc.nextLine().trim();
        if (!mobile.matches("\\d{10}")) {
            System.out.println("Invalid Mobile Number. Must be 10 digi
            return;
        }

        String department = sc.nextLine().trim().replaceAll("\\s+", "
        String doctor = sc.nextLine().trim().replaceAll("\\s+", " ");

        String properName = toProperCase(name);

        Patient p = new Patient(id, properName, mobile, department, do

        patientIds.add(id);
        patientMap.put(id, p);

        deptCount.put(department, deptCount.getOrDefault(department, 0

        System.out.println("Patient Registered Successfully.");
    }

    static void searchPatient(Scanner sc) {
        String id = sc.nextLine().trim();

        if (patientMap.containsKey(id)) {
            Patient p = patientMap.get(id);
            System.out.println("Patient Details");
            System.out.println("ID : " + p.id);
            System.out.println("Name : " + p.name);
            System.out.println("Mobile : " + p.mobile);
            System.out.println("Department : " + p.department);
            System.out.println("Doctor : " + p.doctor);
        } else {
            System.out.println("Patient Not Found");
        }
    }

    static void displayAllPatients() {
        System.out.println("----------------------------------------
        System.out.println("ID Name Department");
        System.out.println("----------------------------------------
        for (String id : patientIds) {
            Patient p = patientMap.get(id);
            System.out.println(p.id + " " + p.name + " " + p.departmen
        }
    }

    static void departmentWiseCount() {
```

```
        for (String id : patientIds) {
            Patient p = patientMap.get(id);
            System.out.println(p.id + " " + p.name + " " + p.departmen
        }
    }

    static void departmentWiseCount() {
        System.out.println("Department Count");
        for (Map.Entry<String, Integer> entry : deptCount.entrySet())
            System.out.println(entry.getKey() + " : " + entry.getValue
        }
    }

    static String toProperCase(String str) {
        String[] words = str.toLowerCase().split(" ");
        StringBuilder result = new StringBuilder();
        for (String word : words) {
            if (word.length() > 0) {
                result.append(Character.toUpperCase(word.charAt(0)))
                      .append(word.substring(1))
                      .append(" ");
            }
        }
        return result.toString().trim();
    }
}

class Patient {
    String id;
    String name;
    String mobile;
    String department;
    String doctor;

    Patient(String id, String name, String mobile, String department,
        this.id = id;
        this.name = name;
        this.mobile = mobile;
        this.department = department;
        this.doctor = doctor;
    }
}
```

Q2 Online Library Book Management System

20 marks

Scenario

Online Library Book Management System

A university library is planning to automate its book management process. The librarian wants a Java application that stores book details, prevents duplicate book entries, allows students to search for books quickly, and generates category-wise reports.

The application should use Java String handling and Java Collection Framework to efficiently manage the library records.

Requirements

Each book contains the following details:

- Book ID
- Book Title
- Author Name
- Category
- Publisher

Use:

- ArrayList to store all book objects.
- HashMap to store Book ID as the key for quick searching.
- HashSet to maintain unique author names.

Menu Driven Program

1. Add Book
2. Search Book by ID
3. Display All Books
4. Display Unique Authors
5. Category-wise Book Count
6. Exit

Your Code

```java
import java.util.*;

public class Main {

    static class Book {
        String bookId;
        String title;
        String author;
        String category;
        String publisher;

        Book(String bookId, String title, String author, String catego
            this.bookId = bookId;
            this.title = title;
            this.author = author;
            this.category = category;
            this.publisher = publisher;
        }
    }

    static String toTitleCase(String s) {
        s = s.trim().toLowerCase();
        if (s.isEmpty()) return s;

        StringBuilder sb = new StringBuilder();

        String[] parts = s.split("\\s+");
        for (int i = 0; i < parts.length; i++) {
            String w = parts[i];
            if (w.length() == 0) continue;
            sb.append(Character.toUpperCase(w.charAt(0))).append(w.sub
            if (i < parts.length - 1) sb.append(" ");
        }
        return sb.toString();
    }

    static String readNonEmpty(Scanner sc) {
        String s = sc.nextLine().trim();
        while (s.isEmpty()) {
            s = sc.nextLine().trim();
        }
        return s;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

Input

```
1
B101
Java Programming
Herbert Schildt
```

Output

```
Book Added Successfully.
Book Added Successfully.
------------------------------
------------------------------
Book ID Title Author
--------------------------------
```

Visible Test Cases

Test Case 1

Test Case 2

Test Case 3

0.00 / 20 marks

```
91
92              } else if (choice == 2) {
93
94                  String id = sc.nextLine().trim();
95
96                  Book b = bookById.get(id);
97                  if (b != null) {
98                      System.out.println("Book Details");
99                      System.out.println();
100                     System.out.println("Book ID : " + b.bookId);
101                     System.out.println("Title : " + b.title);
102                     System.out.println("Author : " + b.author);
103                     System.out.println("Category : " + b.category);
104                     System.out.println("Publisher : " + b.publisher);
105                 } else {
106                     System.out.println("Book Not Available");
107                 }
108
109             } else if (choice == 3) {
110                 System.out.println("----------------------------------
111                 System.out.println("Book ID Title Author");
112                 System.out.println("----------------------------------
113
114                 for (Book b : books) {
115                     System.out.println(b.bookId + " " + b.title + " "
116                 }
117
118             } else if (choice == 4) {
119                 System.out.println("Unique Authors");
120                 System.out.println();
121                 for (String a : uniqueAuthors) {
122                     System.out.println(a);
123                 }
124
125             } else if (choice == 5) {
126                 System.out.println("Category-wise Count");
127                 System.out.println();
128                 for (Map.Entry<String, Integer> e : categoryCount.entr
129                     System.out.println(e.getKey() + " : " + e.getValue
130                 }
131
132             } else if (choice == 6) {
133                 break;
134
135             } else {
136                 System.out.println("Invalid choice. Please enter a num
137             }
138         }
139     }
```